EXPERIMENT 9

Aim: To implement distributed shared memory

Objective: Implement distributed shared memory using Semaphores

Theory:

A semaphore lock on the shared memory buffer reference allows processes accessing the shared memory to prevent a daemon from writing to the memory segment currently being accessed.

Semaphores are a synchronization mechanism used to coordinate the activities of multiple processes in a computer system. They are used to enforce mutual exclusion, avoid race conditions and implement synchronization between processes.

Semaphores provide two operations: wait (P) and signal (V). The wait operation decrements the value of the semaphore, and the signal operation increments the value of the semaphore.

The algorithm is as follows:

```
                    semaphore s = 1;

        Process 1 :                      Process 2 :
while (True)                     while (True)
{   nc1: /* non critical         {   nc2: /* non critical
           section */ ;                    section */
         wait(s);                         wait(s);
  crit1: /* critical section */ ;    crit2: /* critical section */
         signal(s);                       signal(s)
}                                }
```

Code and output:

```
import multiprocessing
import time

# Shared memory and semaphore
shared_memory = multiprocessing.Value('i', 0)  # Shared integer
semaphore = multiprocessing.Semaphore(1)  # Binary semaphore (mutex)

def process_task(process_id):
    global shared_memory

    print(f"Process {process_id} is waiting to access shared memory...")

    # Acquire the semaphore (lock)
    semaphore.acquire()

    try:
        print(f"Process {process_id} acquired lock.")

        # Read the shared value
        current_value = shared_memory.value
        print(f"Process {process_id} read value: {current_value}")

        # Simulate computation
        time.sleep(1)

        # Modify the shared value
```

```
        shared_memory.value = current_value + 1
        print(f"Process {process_id} updated value to: {shared_memory.value}")

    finally:
        # Release the semaphore (unlock)
        print(f"Process {process_id} released lock.")
        semaphore.release()

# Number of processes
num_processes = 4
processes = []

# Create and start multiple processes
for i in range(num_processes):
    p = multiprocessing.Process(target=process_task, args=(i,))
    processes.append(p)
    p.start()

# Wait for all processes to finish
for p in processes:
    p.join()

print(f"Final value in shared memory: {shared_memory.value}")
```

```
Process 0 is waiting to access shared memory...
Process 0 acquired lock.
Process 0 read value: 0
Process 1 is waiting to access shared memory...
Process 2 is waiting to access shared memory...
Process 3 is waiting to access shared memory...
Process 0 updated value to: 1
Process 0 released lock.
Process 1 acquired lock.
Process 1 read value: 1
Process 1 updated value to: 2
Process 1 released lock.
Process 2 acquired lock.
Process 2 read value: 2
Process 2 updated value to: 3
Process 2 released lock.
Process 3 acquired lock.
Process 3 read value: 3
Process 3 updated value to: 4
Process 3 released lock.
Final value in shared memory: 4


...Program finished with exit code 0
Press ENTER to exit console.
```

Conclusion:
Semaphores guarantee mutual exclusion by allowing only one process to access the shared resource at a time using the acquire() and release() operations. If a process holds the semaphore, others must wait until it is released, preventing race conditions and ensuring safe concurrent access.